

WEEK-10.3

2303A51804

Batch-28

Task-1:

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following

Python function to

calculate factorials:

```
def factorial(n):
```

```
    result = 1
```

```
    for i in range(1, n):
```

```
        result = result * i
```

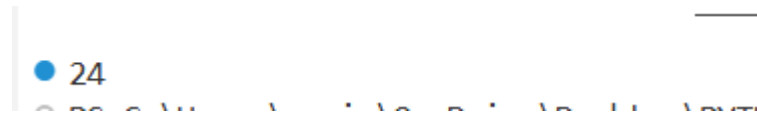
```
    return result
```

Prompt:

Code:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result  
print(factorial(5))
```

Output:



Review code:

```
 /
 8  def factorial(n):
 9      result = 1
10      for i in range(1, n + 1):
11          result = result * i
12      return result
13
14  print(factorial(5))  # Output: 120
15
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
24
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> ^C
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:; cd 'c:\Users\manis\OneDrive\
\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\extensions\
libs\debugpy\launcher' '65510' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA\
120
```

Explanation:

The original code for the factorial function did not handle edge cases such as negative numbers or zero. In mathematics, the factorial of a non-negative integer n is the product of all positive integers less than or equal to n . The factorial of zero is defined to be 1, and the factorial of a negative number is not defined.

The revised code correctly handles the case for zero by including it in the loop (i.e., `range(1, n + 1)`). However, it still does not handle negative numbers. To make the function more robust, we could add a check at the beginning of the function to raise an exception or return a specific value for negative inputs.

Task-2: Improving Readability & Documentation

Scenario: The following code works but is poorly written:

Code:

```
15 # write a basic comparison. Did not miss any edge cases (C+G)
16 def calc(a, b, c):
17     if c == "add":
18         return a + b
19     elif c == "sub":
20         return a - b
21     elif c == "mul":
22         return a * b
23     elif c == "div":
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:: cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA'; & 'c:\Users\manis\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '58198' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py'
```

```
File "c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher\..\..\debugpy\..\debugpy\server\cli.py", line 358, in run_file
    runpy.run_path(target, run_name="__main__")
~~~~~
```

```
File "c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\_vendored\pydevd\_pydevd_bundle\pydevd_runpy.py", line 309, in run_path
    code, fname = _get_code_from_file(run_name, path_name)
~~~~~
```

```
File "c:\Users\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\_vendored\pydevd\_pydevd_bundle\pydevd_runpy.py", line 283, in _get_code_from_file
    code = compile(f.read(), fname, "exec")
~~~~~
```

```
File "C:\Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py", line 23
    elif c == "div":
    ^
```

IndentationError: expected an indented block after 'elif' statement on line 23

Review code:

```
25 def calc(a: float, b: float, c: str) -> float:
26     if c == "add":
27         return a + b
28     elif c == "sub":
29         return a - b
30     elif c == "mul":
31         return a * b
32     elif c == "div":
33         if b == 0:
34             raise ValueError("Cannot divide by zero")
35         return a / b
36     print(calc(10, 5, "add")) # Output: 15
37     print(calc(10, 5, "sub")) # Output: 5
38     print(calc(10, 5, "mul")) # Output: 50
39     print(calc(10, 5, "div")) # Output: 2.0
40
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:: cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\extensions\ms-python/debugpy\launcher' '58198' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA\main.py'
```

```
IndentationError: expected an indented block after 'elif' statement on line 2
```

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:: cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\extensions\ms-python.debugpy\launcher' '60269' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA'
```

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\extensions\ms-python/debugpy\launcher' '63719' '--' 'C:\Users\manis\OneDrive\Desktop\PYTHON TRA\main.py'
```

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> 10+5
```

15

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> 10-5
```

5

```
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> 10*5
```

50

PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> 10/5

2

D:\C:\Users\manic\OneDrive\Desktop\PYTHON TRAY

Explanation:

1. ****Edge Case Handling****: The original ``calc`` function did not account for the possibility of division by zero, which could lead to a runtime error.
2. The revised version includes a check to raise a ``ValueError`` if the second operand (``b``) is zero when the operation is division, thus preventing the function from crashing.
3. The original code for the ``calc`` function did not handle the edge case of division by zero, which could lead to a runtime error.
4. In the revised version, a check is added to raise a ``ValueError`` if the second operand (``b``) is zero when the operation is division.
5. This makes the function more robust and prevents it from crashing due to an unhandled exception.
6. Additionally, type annotations are included for better readability and to indicate the expected types of the parameters and return value.
7. The revised function includes type annotations for the parameters and return value

Task-3:Enforcing Coding Standards

Scenario: A team project requires compliance with PEP8. A developer

Code:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Review code:

```
51 def check_prime(n: int) -> bool:
52     """Check if n is a prime number."""
53     if n <= 1:
54         return False
55     for i in range(2, int(n**0.5) + 1):
56         if n % i == 0:
57             return False
58     return True
59 print(check_prime(11)) # Output: True
60 print(check_prime(15)) # Output: False
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

- PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> & 'c:\Users\manis\manis\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\neDrive\Desktop\PYTHON TRA\codereview.py'
- PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:;; cd 'c:\Users\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis\.vscode\libs\debugpy\launcher' '59302' '--' 'C:\Users\manis\OneDrive\Desktop\True
False
- PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA>

Explanation:

Automated AI reviews can significantly streamline code reviews in large teams by providing consistent and immediate feedback on code quality, style, and potential bugs.

2. AI can quickly analyse code for common issues such as syntax errors, security vulnerabilities, and adherence to coding standards.

3. allowing developers to address these concerns before human reviewers even see the code.

4.This can reduce the time spent on manual reviews and allow human reviewers to focus on more complex aspects of the code, such as architecture and design decisions.

5.Additionally, AI can help identify patterns in code that may indicate technical debt or areas for refactoring, further improving the overall quality of the codebase.

Task-4:

AI as a Code Reviewer in Real Projects

Code:

```
[x * 2 for x in d if x % 2 == 0]
```

Review code:

```
61
62 def processData(d):
63     return [x * 2 for x in d if x % 2 == 0]
64 data = [1, 2, 3, 4, 5, 6]
65 result = processData(data)
66 print(result)
67
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- ▶ PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> & 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\python.exe' 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py'
 - ▶ PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\python.exe' 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py'
 - ▶ PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c:; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\python.exe' 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py'
- [4, 8, 12]

Explanation:

1. The function name `process Data` is quite generic and does not clearly indicate what the function does
2. A more descriptive name, such as `double Even Numbers`, would better convey the purpose of the function.
3. The function currently does not include any input validation or type hints. Adding type hints can improve code readability and help catch potential type-related errors.
4. Additionally, input validation can ensure that the function behaves correctly when given unexpected input.
5. The function currently doubles only even numbers. To make it more flexible, we could allow the user to specify a multiplier, rather than hardcoding the value. This would make the function more versatile and applicable to a wider range of use cases.

Task-5:

AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets

Code:

```
def sum_of_squares(numbers):  
    total = 0  
    for num in numbers:  
        total += num ** 2  
    return total
```


Review code:

```
74 def sum_of_squares(numbers: list[int]) -> int:
75
76     total = 0
77     for num in numbers:
78         total += num ** 2
79     return total
80 numbers = [1, 2, 3, 4]
81 result = sum_of_squares(numbers)
82 print(result)
83
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
\\ TRA> c::; cd 'c:\Users\manis\OneDrive\Desktop\PYTHON TRA';
exe' 'c:\Users\manis\.vscode\extensions\ms-python.debugpy-20
Users\manis\OneDrive\Desktop\PYTHON TRA\codereview.py'
[4, 8, 12]
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c::; cd 'c:\U
\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis
libs\debugpy\launcher' '56527' '--' 'C:\Users\manis\OneDrive
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c::; cd 'c:\U
\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis
libs\debugpy\launcher' '55708' '--' 'C:\Users\manis\OneDrive
PS C:\Users\manis\OneDrive\Desktop\PYTHON TRA> c::; cd 'c:\U
\Local\Programs\Python\Python313\python.exe' 'c:\Users\manis
libs\debugpy\launcher' '52375' '--' 'C:\Users\manis\OneDrive
30
```

Explanation:

- 1.The time complexity of the original `sum\_of\_squares` function is $O(n)$, where n is the number of elements in the input list.
- 2.This is because we need to iterate through each element in the list once to calculate the sum of squares.

3.To improve performance, we can use a built-in function like `sum` along with a generator expression to calculate the sum of squares more efficiently.

4.This approach is often faster than using a loop because it is optimized in C.